开源项目架构附录

陈星强 · github.com/chenxingqiang · 每页包含项目介绍与系统架构

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

CONTENTS · one architecture per page

TraceSmith
GPU profiling & replay

YiRage
RuntimeFusion compiler

JAX-sklearn
transparent JAX acceleration

Secret-Learn
privacy-preserving ML

YouTube-DNN
two-tower recall

Research CLI
agent workflow

Questions-Gen
three-stage training

阅读约定

自上而下读取主数据流；虚线框表示架构分区；盒内仅列模块与职责。

蓝色实线表示主链路，紫色虚线表示回退或反馈，不以源码文件充当架构层。

TraceSmith — 系统架构 · GPU Trace 端到端数据流

Application → bindings → capture / state / format → replay & export → GPU backends

主数据流　虚线框 = 架构分区
回退 / 反馈　盒内 = 模块 + 职责

项目介绍

TraceSmith 是面向 AI 编译器、深度学习框架与 GPU 驱动工程师的跨平台性能分析和确定性回放系统。
C++17 核心以低开销采集执行事件，写入紧凑 SBT 轨迹，并导出时间线用于分析、调试与问题复现。

APPLICATION · 用户入口

- **C++ / Python Application** — 嵌入业务进程
- **CLI** — profile · record · view · export
- **External UI** — Perfetto · Chrome

BINDINGS · Python / C++ 边界

- **_tracesmith.so** — pybind11 native extension

CORE LIBRARY · C++17

- **Capture Module** — CUPTIProfiler · MCPTIProfiler; Metal · Ascend · ROCm · eBPF
- **Common + Format** — RingBuffer SPSC · StackCapture; SBTWriter / SBTReader
- **State Module** — GPUStateMachine; TimelineBuilder · PerfettoExporter
- **Replay Module** — ReplayEngine · StreamScheduler; DeterminismChecker · FrameCapture

CROSS-CUTTING · 集群与调试

replay / debug feedback

- **Cluster Module** — GPUTopology · TimeSync; NCCLTracker
- **GDB / Tracy** — RSPHandler · GPUDebugEngine; TracyExporter

GPU BACKENDS · 系统接口

- **NVIDIA** — CUDA · CUPTI · nsys
- **MetaX** — MACA · MCPTI · mcTracer
- **Apple / AMD** — Metal · xctrace · ROCm
- **Huawei / Linux** — CANN · msprof · eBPF

OUTPUTS · Trace 产物

- **trace.sbt** — compact binary
- **trace.json / .perfetto** — Perfetto timeline
- **graph.dot / ASCII** — analysis views

主链：GPU event → lock-free ring → SBT/state → replay 或 Perfetto；Cluster / GDB 为旁路能力。

YiRage (YPK) — 系统架构 · RuntimeFusion 数据流

Inference engine owns scheduling/KV; RF owns fusion identity; search and execute use the same backend

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

YiRage 是面向大模型推理的多后端算子编译与运行时系统，核心能力是计算图超优化和 RuntimeFusion。

系统离线搜索并验证后端特定融合计划，再将 FusionCapsule 注入 vLLM 或 SGLang 的逐步解码流程。

INFERENCE ENGINE · 调度与 KV 所有者

vLLM
continuous batch · PagedAttention

SGLang
RadixAttention · extend meta

Decode Metadata
block table · seq lens · SM budget

RUNTIMEFUSION · 每个 decode step

RF.step(inputs, meta)
select 0..N capsules

FusionCapsule
MLP gate_up + down
segment pipeline

Native Fallback
skip → engine native ops

SEARCH & PLAN · offline / warm-up

superoptimize(backend)
KN / TB graph search

FusionPlan / μGraph
profile · fingerprint verify

Persistent Cache
shape bucket restore

Ray Coordinator
optional distributed search

fallback / re-search

BACKEND MANAGER · 编译与执行边界

BackendRegistry
search + execute same backend

KNGraph / TBGraph
transpiler → kernels

yirage.core
Cython runtime

BACKENDS → HARDWARE

CUDA / MACA
NVIDIA GPU · MetaX C500/C550

ROCm / MPS
AMD GPU · Apple Silicon

Ascend / NKI
Huawei 910 · AWS Neuron

CPU / MLIR / Triton
x86 · ARM · extra targets

主链：Engine metadata → RF.step → FusionCapsule → BackendRegistry → target hardware。

JAX-sklearn (xlearn) — 五层架构 · 透明加速数据流

100% sklearn API → runtime injection → proxy dispatch → array conversion → JAX hardware

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

JAX-sklearn 在保持 scikit-learn 调用接口不变的前提下，透明地以 JAX 加速受支持的估计器。
运行时代理依据算法、数据规模与硬件平台选择 JAX 或原生 sklearn，并保持输入输出兼容。

USER CODE · sklearn-compatible API

import xlearn as sklearn
LinearRegression · KMeans · PCA

Estimator Calls
fit · predict · transform

COMPATIBILITY · xlearn/__init__.py

__getattr__
lazy module loading

_auto_jax_accelerate_module
runtime class injection

AcceleratorRegistry
algorithm implementation map

JAX ACCELERATION · xlearn/_jax

EstimatorProxy
fit-time dispatch

Universal JAX
JIT · vmap · accelerated estimator

sklearn Fallback
original estimator

graceful fallback

DATA MANAGEMENT · boundary conversion

NumPy → JAX
dtype · shape normalization

Policy / Thresholds
dataset size · platform · config

JAX → NumPy
sklearn-compatible outputs

HARDWARE ABSTRACTION · JAX backend

CPU
jaxlib

CUDA GPU
JAX XLA

TPU
JAX XLA

Apple Metal
experimental

代理根据算法、数据规模和配置选择 JAX；不满足条件或执行失败时回退原生 sklearn。

Secret-Learn — 六层架构 · 隐私计算数据流

sklearn-compatible API → privacy mode → JAX / SecretFlow compute → secure devices

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

Secret-Learn 以 sklearn 兼容接口提供联邦学习、秘密分享和拆分学习三种隐私机器学习模式。

系统结合 JAX 加速与 SecretFlow 安全设备，使工作流能够明确控制明文、秘密份额和中间激活的驻留位置。

APPLICATION · 多方隐私数据

- **Healthcare / Finance** — cross-party modeling
- **IoT / Research** — distributed learning

SKLEARN API · 统一算法入口

- **secretlearn.*** — fit · predict · transform
- **algorithm_classifier.py** — classify fit signature / epochs

PRIVACY MODES · execution placement

- **FEDERATED LEARNING** — PYU local compute; HEU secure aggregation
- **SECRET SHARING** — SPU MPC · ABY3 / CHEETAH
- **SPLIT LEARNING** — split model · encrypted activations

ACCELERATION & COMPUTE

- **JAX Acceleration** — JIT · vmap · xlearn
- **SecretFlow Runtime** — party orchestration

SECURE DEVICES

- **PYU** — party-local plaintext
- **HEU** — homomorphic aggregation
- **SPU** — secure MPC
- **TEE** — optional enclave

INFRASTRUCTURE & HARDWARE

- **SecretFlow** — distributed secure runtime
- **JAX / XLA** — CPU · GPU · TPU
- **sklearn** — API compatibility

同一 sklearn 调用由 FL / SS / SL 决定数据驻留与执行设备；JAX 加速可公开的本地计算。

YouTube-DNN-RecSys — 系统架构 · 召回数据流

Python / Spark data preparation → TFRecords → dnn.py → user & item vectors → prediction

主数据流　　虚线框 = 架构分区
回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

YouTube-DNN-RecSys 使用 TensorFlow 与 Spark 实现 YouTube 推荐系统中的候选召回阶段。
系统将用户行为历史转换为 TFRecords，训练用户与物品向量，并为在线服务生成 Top-K 召回候选。

DATA PROCESSING · 离线样本构建

Python Pipeline
data2tfrecords.py
read_tfrecords.py

Spark Pipeline
MakeDataOne.scala
MakeDataTwo.scala

FeatureBuilder
history + continuous features

STORAGE · 训练与模型产物

TFRecords
train / evaluation

Checkpoints
ckpt / eval / modelpath

MODEL TRAINING · src/python/models/dnn.py

Input
visit_items_index × 5
continuous_features × 16

Item Embedding
500 × 16 · average pooling

User Tower
Dense 64 → 32 → vector 16

Softmax Output
500 classes · Top-K

evaluation feedback

PREDICTION · serving paths

Python SavedModel
load_dnn_model.py

User Vector
PredictUserVector.scala

Item Table
ItemEmbeddingPredictor.scala

RECOMMENDATION OUTPUT

Candidate Recall
user vector × item embedding

Top-K Items
next-item recommendation

主链：行为历史 → TFRecords → embedding mean-pool → user vector → item table matching → Top-K。

Research CLI — 系统架构 · Agent 请求数据流

Terminal UI → core orchestration & routing → tools / MCP / research modules → model providers

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

Research CLI 是终端优先的 AI Agent 工作流，将交互会话连接到模型、工作区工具和 MCP 服务。

核心路由负责上下文与模型供应商选择，研究模块覆盖论文写作、文献管理、数据分析和投稿准备。

CLI FRONTEND · packages/cli

Interactive TUI
research.tsx · sessions

Commands
mcp · skills · hooks

ACP
acpSession.ts

CORE ORCHESTRATION · packages/core

Context & Memory
config.ts · memory.ts

modelRouterService
provider selection

Routing Strategy
default · classifier · approval

TOOLS & INTEGRATIONS

Workspace Tools
filesystem · shell

MCP
external service adapters

Research Modules
writing · bibliography · analysis

SDK / A2A / VS Code
embedded clients

tool result loop

MODEL PROVIDERS · provider config

Gemini / Vertex
hosted models

OpenAI / Anthropic
hosted models

Baidu / DeepSeek / Qwen
hosted models

RESULT STREAM

Tool Results
structured observations

Assistant Response
streamed to terminal

主链：prompt → packages/cli → packages/core router → provider / tools → structured result → terminal。

Questions-Gen — 系统架构 · 三阶段训练数据流

Training data → SFT → GRPO → DeepSeek distillation → validation → Hugging Face / Ollama

主数据流　　虚线框 = 架构分区

回退 / 反馈　　盒内 = 模块 + 职责

项目介绍

Questions-Gen 是基于 Qwen3-14B 的数学竞赛题生成与评估系统，采用分阶段训练策略。

训练链路结合监督微调、GRPO 奖励优化、DeepSeek-R1 蒸馏、质量验证以及 Hugging Face 和 Ollama 部署。

TRAINING DATA · problem mixture

- **Historical Problems** — 50% source distribution
- **Variants** — 30% conditional variation
- **Novel Problems** — 20% innovation target

Stage 1 SFT · QuestionsGenTrainer.py

- **Qwen3-14B** — supervised fine-tuning
- **Stage-1 Checkpoint** — stage1-fp-merged

Stage 2 GRPO · reward optimization

- **RewardCalculator** — 0.4 difficulty · 0.3 novelty · 0.2 rigor · 0.1 diversity
- **NoveltyConstraint** — variation / duplication guard
- **GRPO Trainer** — group-relative optimization

Stage 3 Distill · teacher supervision

quality feedback

- **DeepSeekTeacher** — DeepSeek-R1 reasoning teacher
- **Final Model** — final-merged-16bit

VALIDATION

- **ModelValidator** — single checkpoint
- **BatchValidator** — algebra · geometry · calculus
- **QualityEvaluator** — clarity · difficulty · originality

DEPLOYMENT

- **Hugging Face** — three published stages
- **Ollama** — local inference
- **questions-gen CLI** — validate · batch · quality

主链：混合题库 → SFT → GRPO reward/novelty → DeepSeek-R1 distill → validate → publish。